

《 Introduction to Embedded System 》

Lab-5

Major: Software Engineering

Name: Md Tasnimul Hasan

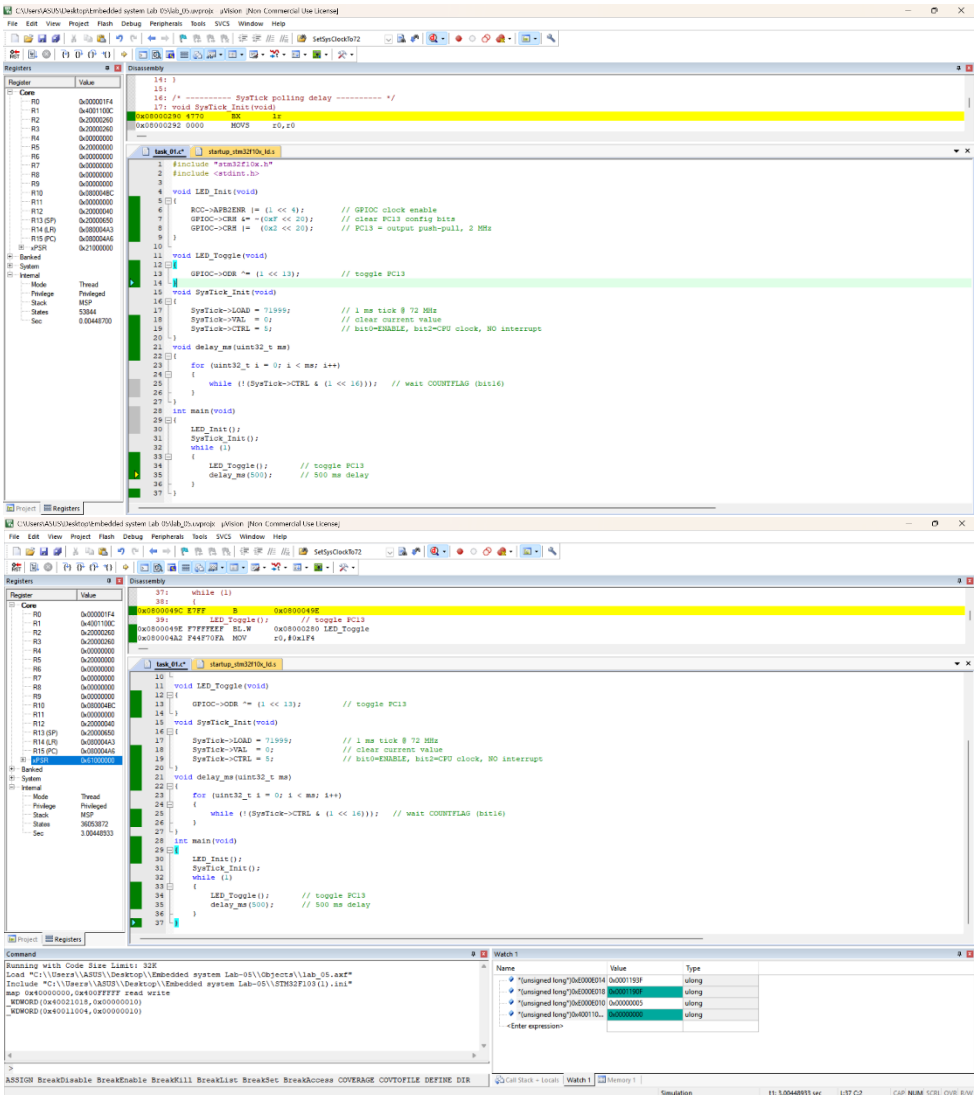
Student ID: 202480090104

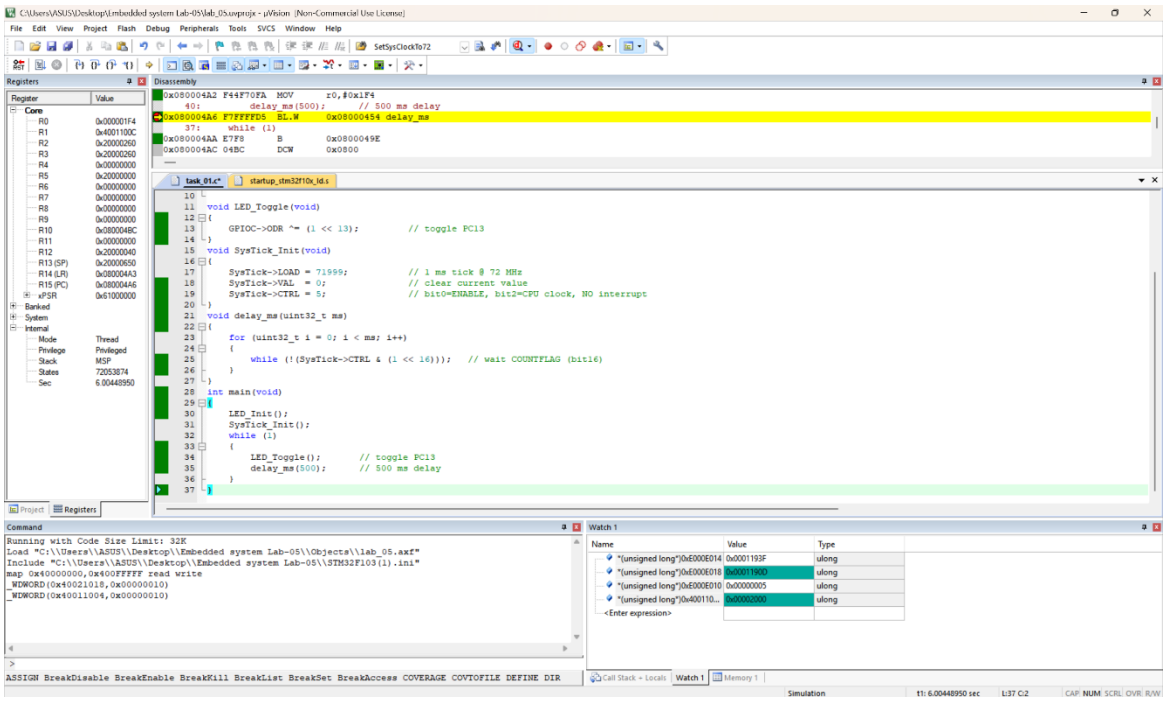
Task 1: SysTick Polling Delay and 500 ms LED Blink

Objective: To configure the Cortex-M3 SysTick timer for a 1 ms time base at 72 MHz using the polling method (without interrupts) and to verify the timing by toggling the PC13 LED every 500 ms.

Description: Configured the SysTick timer with a LOAD value of 71999 to generate a 1 ms tick at a 72 MHz CPU clock. The CTRL register was set to 0x05 to enable the timer without interrupts, and the delay_ms() function was implemented by polling the COUNTFLAG bit. Verified via the Watch window that LOAD reads 0x0001193F and GPIOC ODR toggles correctly. The Registers window confirmed that the simulated time (Sec) increases by exactly 0.5 seconds between breakpoints.

Code and Output:



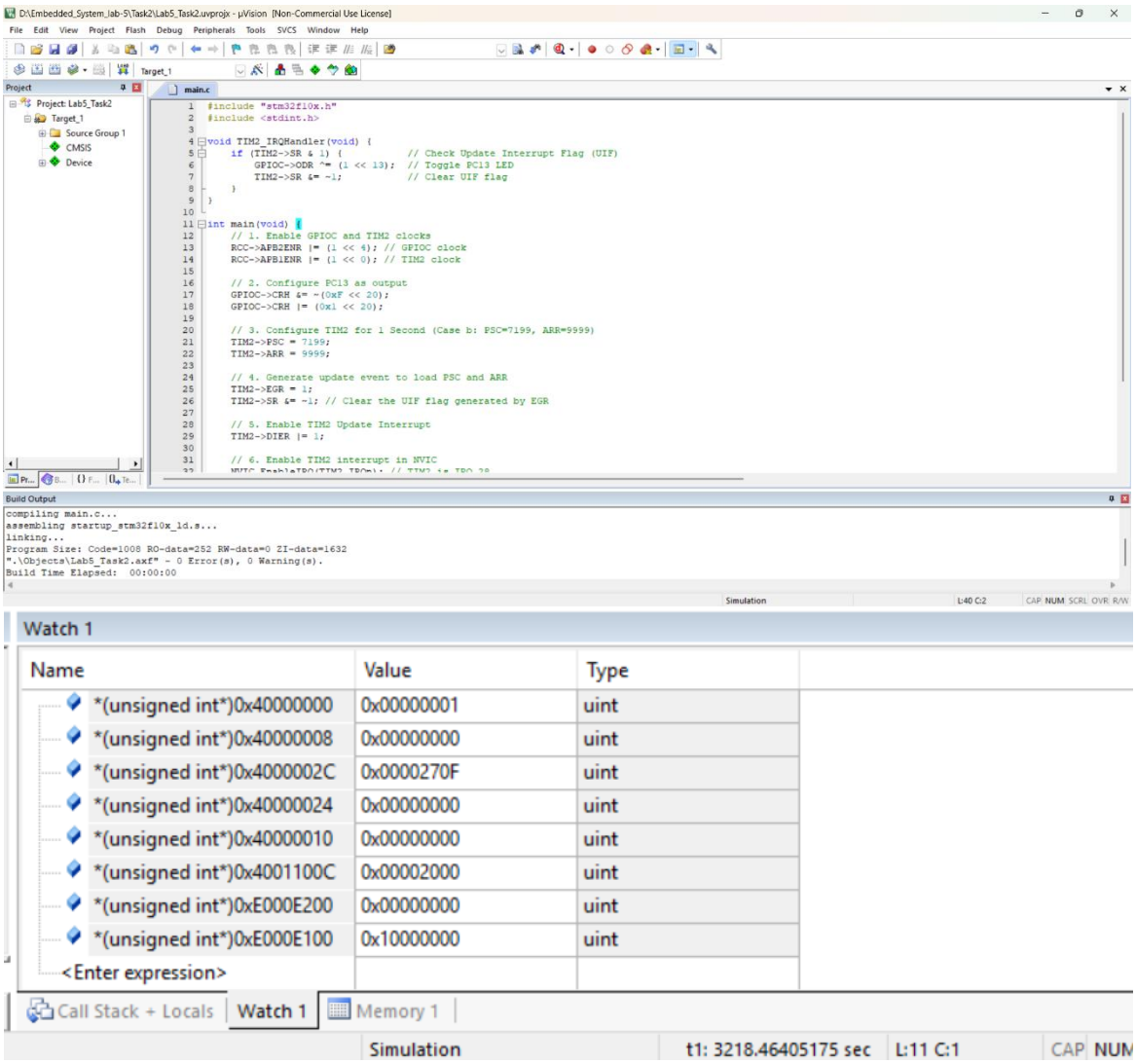


Task 2: TIM2 Polling Delay (1s and 0.5s)

Objective: To configure General-Purpose Timer 2 (TIM2) in up-counting mode to generate precise time delays (1 second and 0.5 seconds) and toggle the LED using the Update Interrupt Flag (UIF) in a polling loop.

Description: Enabled the TIM2 clock and calculated the Prescaler (PSC) and Auto-Reload Register (ARR) values. For a 1-second delay (Case b), PSC was set to 7199 and ARR to 9999, and for a 0.5-second delay (Case c), ARR was changed to 4999. The timer was started using the CEN bit in CR1, and the main loop waits for the UIF flag in TIM2->SR to toggle the LED. Watch window verification confirmed that the CNT register counts up and the simulated time difference matches the calculated periods.

Code and Output:

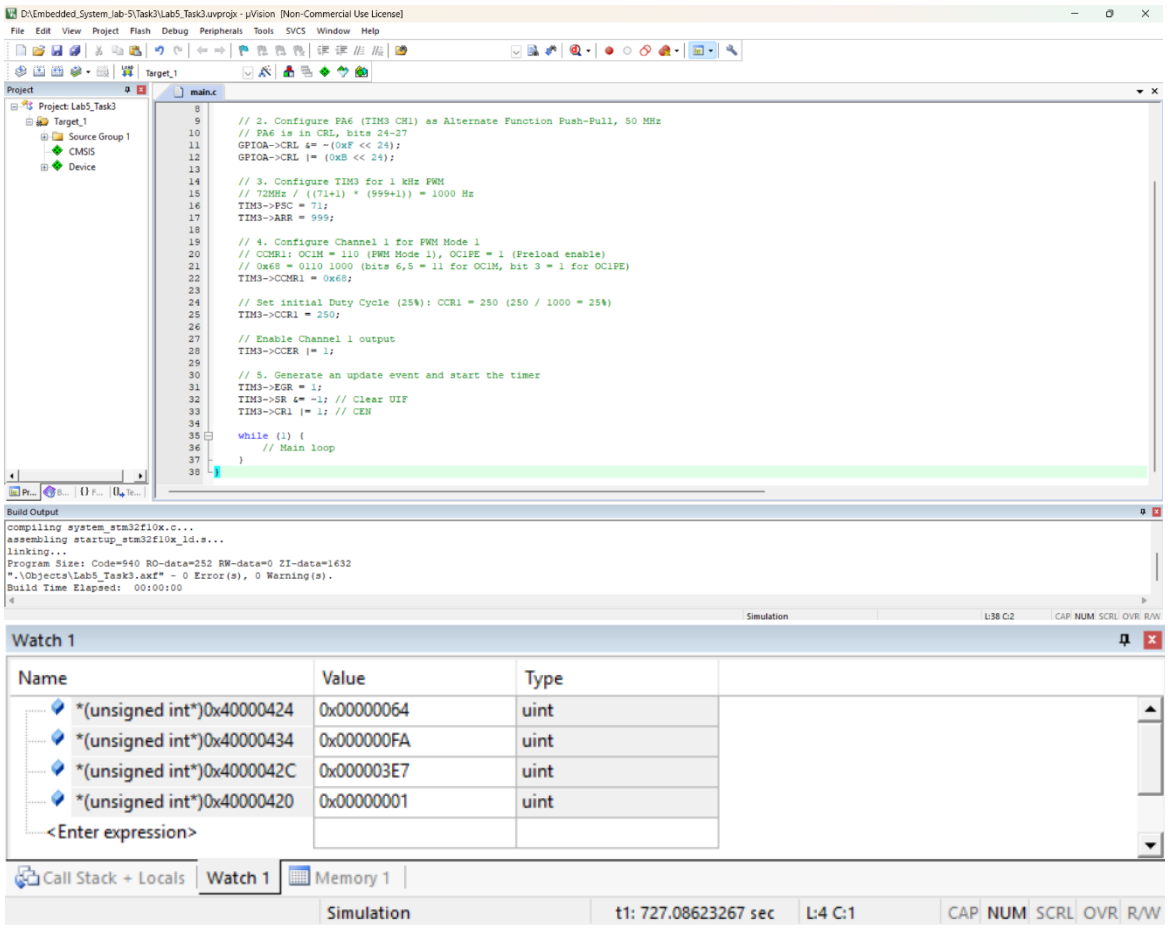


Task 3: TIM3 PWM (PA6) and LED Fade Effect

Objective: To configure TIM3 Channel 1 to generate a 1 kHz PWM signal on pin PA6 (alternate function push-pull) and to dynamically adjust the duty cycle to create a fading LED effect.

Description: Configured PA6 as an alternate-function push-pull output and set up TIM3 with PSC=71 and ARR=999 to achieve a 1 kHz frequency. Channel 1 was set to PWM Mode 1 via CCMR1 with an initial duty cycle of 25% (CCR1 = 250). By reusing the 1 ms SysTick delay, the code increments and decrements the CCR1 value by 10 every 10 ms to create a smooth fade effect from 0% to 100%. The Watch window and Logic Analyzer (if available) verified the duty cycle changes and pulse widths.

Code and Output:

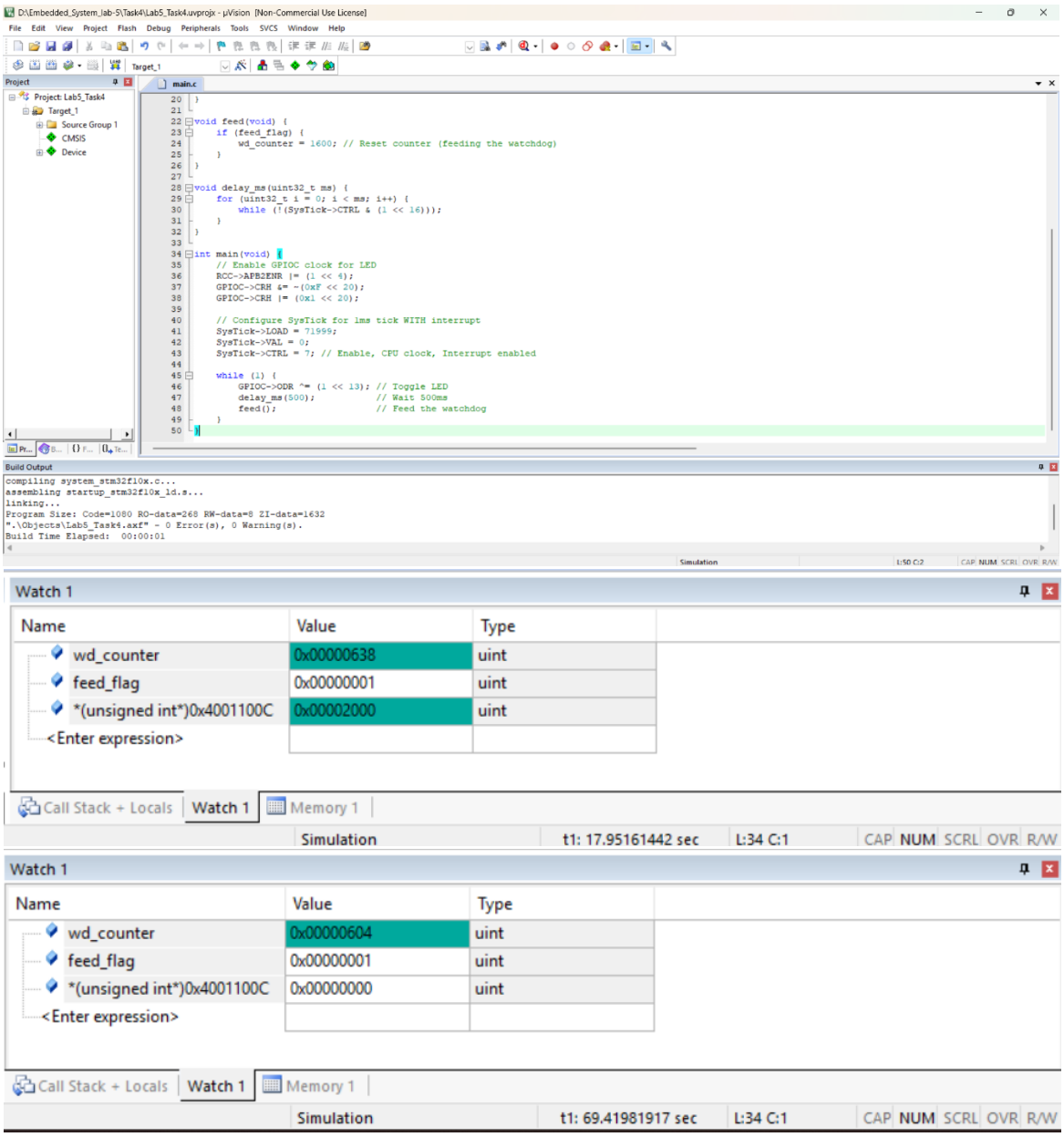


Task 4: Independent Watchdog (IWDG)

Objective: To configure the Independent Watchdog (IWDG) using the internal LSI clock, calculate the timeout periods, and verify the system reset behavior when the watchdog is not fed.

Description: Enabled the LSI clock, unlocked the IWDG, and set the Prescaler to 4 (divide by 64) and RLR to 1000, which yielded a timeout period of roughly 1 second depending on the LSI tolerance (30-60 kHz). In Run A, the watchdog was successfully fed every 500 ms by writing 0xAAAA to the KR register, preventing a reset. In Run B (without feeding), the system was expected to reset after the timeout, demonstrating the watchdog's safety mechanism.

Code and Output:



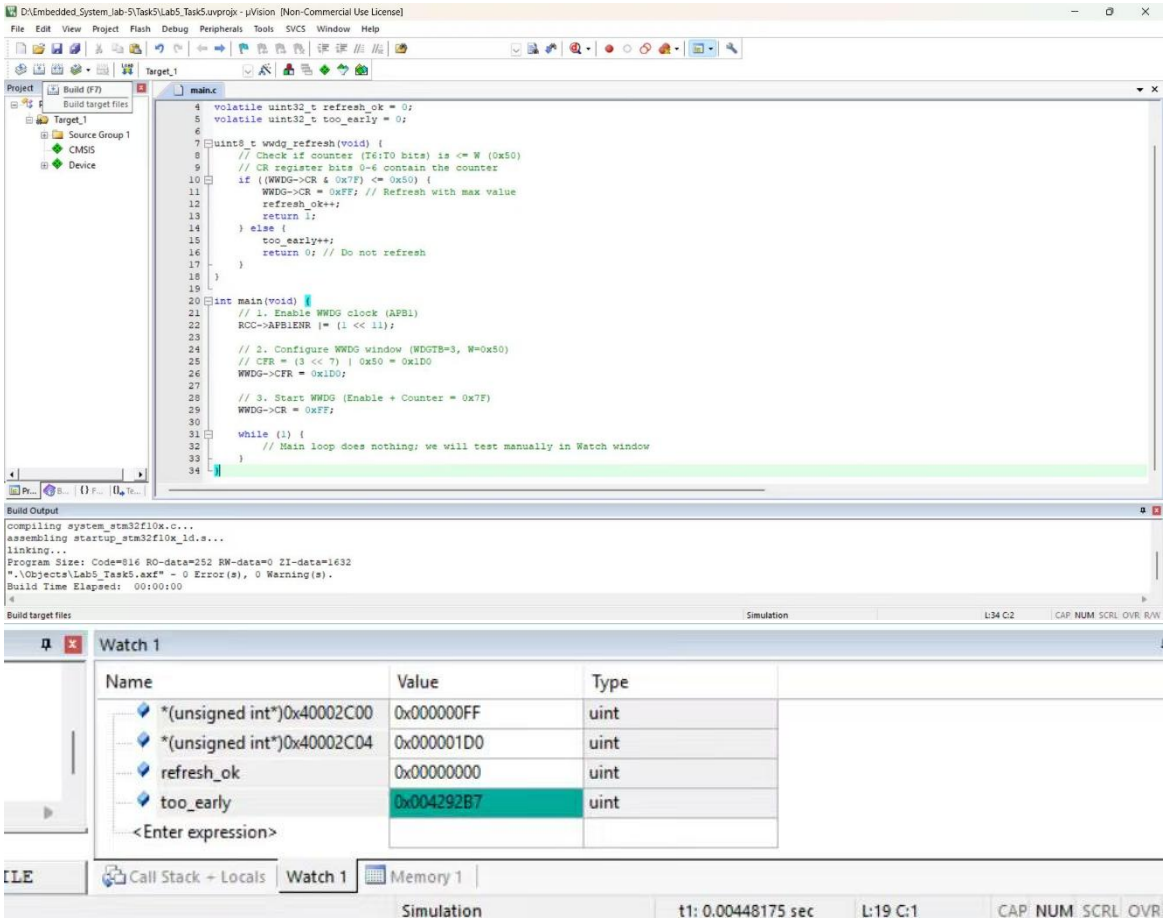
Task 5: Window Watchdog (WWDG)

Objective: To configure the Window Watchdog (WWDG) with a specific time window, calculate the time per counter step, and test the "too early" and "too late" refresh conditions.

Description: Calculated the WWDG step time using PCLK1 (36 MHz) and WDGTB=3, resulting in ~0.91 ms per step. The window value was set to 0x50 and the counter started at 0x7F, creating a safe refresh window between 42.8 ms and 58.2 ms. Tested three cases: refreshing too early (CR=0xFF), refreshing inside the window (CR=0xCA), and letting the

counter drop below the threshold (CR=0xBF). The wwdg_refresh() function successfully blocked early refreshes, mimicking the hardware reset behavior.

Code and Output:



Conclusion

In this lab, various timer interfaces of the STM32F103C4 microcontroller were successfully configured and verified using the Keil MDK simulator. The SysTick timer was utilized to generate precise 1 ms polling delays, while General-Purpose Timers (TIM2 and TIM3) were configured for longer time bases and PWM signal generation. The lab demonstrated how Prescaler (PSC) and Auto-Reload Register (ARR) values dictate timer frequencies and overflow periods. Furthermore, the implementation of PWM mode on TIM3 proved effective in controlling LED brightness via duty cycle adjustments. Finally, the hardware safety mechanisms were explored by configuring the Independent Watchdog (IWDG) on the LSI clock and the Window Watchdog (WWDG) on the APB1 clock, highlighting their crucial roles in recovering from software failures. Overall, the Watch and Registers windows in the simulator provided a clear, step-by-step verification of timer registers, interrupt flags, and simulated timing, reinforcing the theoretical concepts with practical implementation.